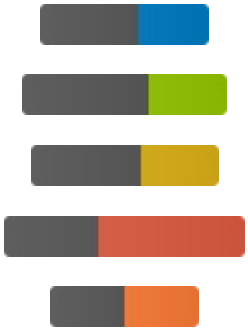
LeHome Challenge 2026

Challenge on Garment Manipulation Skill Learning in Household Scenarios

Competition Website



⚠ Important Reminder

During evaluation, garments from different categories will be loaded randomly.

Participants cannot access the ground-truth garment category labels in the simulator during evaluation.

Therefore, if your policy requires garment labels, you must design your own method to obtain them (e.g., train a classifier).

Please submit a policy that is appropriate for this evaluation protocol.

Table of Contents

Quick Start

> ⚠ ***IMPORTANT**:*
>
> *For Ubuntu version and GPU-related settings, please refer to the IsaacSim 5.1.0 Documentation. And the simulation currently only supports CPU devices.*

1. Installation

We offer two installation methods: UV and Docker for submission and local evaluation.

Use UV

The simulation environment is based on the IssacLab and LeRobot repositories; please refer to UV installation guide.

Use Docker

The simulation environment is based on the IssacLab and LeRobot repositories; please refer to Docker installation guide.

2. Assets & Data Preparation

Download Simulation Assets

Download the required simulation assets (scenes, objects, robots) from HuggingFace:

```
# This creates the Assets/ directory with all required simulation resources
hf download lehome/asset_challenge --repo-type dataset --local-dir Assets
```

Download Example Dataset

We provide demonstrations for four types of garments. Download from HuggingFace:

```
hf download lehome/dataset_challenge_merged --repo-type dataset --local-dir Datasets/example
```

If you need depth information or individual data for each garment. Download from HuggingFace:

```
hf download lehome/dataset_challenge --repo-type dataset --local-dir Datasets/example
```

Collect Your Own Data

For detailed instructions on teleoperation data collection and dataset processing, please refer to our Dataset Collection and Processing Guide (using SO101 Leader is strongly recommended).

3. Train

We provide several training examples; the models and training framework are from LeRobot.

Quick Start

Train using one of the pre-configured training files:

```
lerobot-train --config_path=configs/train_<policy>.yaml
```

Available config files:

- `configs/train_act.yaml` - ACT
- `configs/train_dp.yaml` - DP
- `configs/train_smolvla.yaml` - SmolVLA

Key configuration options:

- **Dataset path**: Update `dataset.root` to point to your dataset
- **Input/Output features**: Specify which observations and actions to use
- **Training parameters**: Adjust `batch_size`, `steps`, `save_freq`, etc.
- **Output directory**: Modify `output_dir` to save models elsewhere

> ***For detailed training instructions, feature selection guide, and configuration options, see our Training Guide.***

4. Eval

Evaluate your trained policy on the challenge garments. The framework supports LeRobot policies and custom implementations.

Examples:

```
# Evaluate using LeRobot policy
# Note: --policy_path and --dataset_root are required parameters for LeRobot policies, ready to run once the dataset and model checkpoints are prepared.
python -m scripts.eval \
    --policy_type lerobot \
    --policy_path outputs/train/act_top_long/checkpoints/last/pretrained_model \
    --garment_type "top_long" \
    --dataset_root Datasets/example/top_long_merged \
    --num_episodes 2 \
    --enable_cameras \
    --device cpu

# Evaluate custom policy
# Note: Participants can define their own model loading logic within the policy class. Provides flexibility for participants to implement specialized loading and inference logic.
python -m scripts.eval \
    --policy_type custom \
    --garment_type "top_long" \
    --num_episodes 5 \
    --enable_cameras \
    --device cpu
```

Common Options

Parameter	Description	Default	Required For
--policy_type	Policy type: lerobot, custom	lerobot	All
--policy_path	Path to model checkpoint	-	All (passed as model_path for custom)
--dataset_root	Dataset path (for metadata)	-	**LeRobot only**
--garment_type	Type of garments: top_long, top_short, pant_long, pant_short, custom	top_long	All
--num_episodes	Episodes per garment	5	All
--max_steps	Max steps per episode	600	All
--save_video	Save evaluation videos		All
--video_dir	Directory to save evaluation videos	outputs/eval_videos	--save_video
--enable_cameras	Enable camera rendering		All
--device	Device for inference: only cpu	'cpu'	All

--headless	Used for evaluation without GUI	disabled	All

Parameter Descriptions:

- **Required for LeRobot Policy**: `--policy_path` (model path) and `--dataset_root` (dataset path, used for loading metadata).
- **Custom Policy**: `--policy_path` is passed to the policy constructor as `model_path`. Participants can define their own model loading logic (refer to `scripts/eval_policy/example_participant_policy.py`).

Garment Test Configuration

Evaluation is performed on the `Release` set of garments. Under the directory `Assets/objects/Challenge_Garment/Release`, each garment category folder contains a corresponding text file listing the garment names (e.g., `Top_Long/Top_Long.txt`).

- **Evaluate a Category**: Set `--garment_type` to `top_long`, `top_short`, `pant_long`, or `pant_short` to evaluate all garments within that category.
- **Evaluate Specific Garments**: Edit `Assets/objects/Challenge_Garment/Release/Release_test_list.txt` to include only the garments you want to test, then run with `--garment_type custom`.

> ***For detailed policy evaluation guide**, see eval_guide.*

Submission

Once you are satisfied with your model's performance, submit your results through this google form. In summary, the submitted files include:

- A **README (.md) file** that provides detailed instructions on how we can evaluate your policy.
- A **docker image URL** or **huggingface repo link** that contains your checkpoints, source code and any other dependencies required for evaluation.
- The .txt file which contains rollout results you performed.
- Source Code File(Optional, but highly encouraged). You can provide a link to your source code repository. This will NOT be used for evaluation, but it allows us to debug things easier if anything goes wrong during evaluation.

> *Please check the google form for more detailed submission information.*

Acknowledgments

This project stands on the shoulders of giants. We utilize and build upon the following excellent open-source projects:

- **Isaac Sim** - For photorealistic physics simulation
- **Isaac Lab** - For modular robot learning environments

- **LeRobot** - For state-of-the-art Imitation Learning algorithms
- **Marble** - For diverse simulation scene generation